

Programowanie w języku Rust

Krystian Piękoś

Informacje organizacyjne

- Czas trwania szkolenia:
 - 9:00 - 16:00
- Materiały szkoleniowe:
 - [repozytorium GIT](#)
- Przerwy
- Ankieta

Crate'y i moduły w Rust

Crate'y i moduły w Rust

- `crate` to pakiet kompilacyjny Rust
 - crate może być binarny lub biblioteczny
- `module` to jednostka organizacyjna w kodzie
 - moduły pomagają organizować przestrzeń nazw i widoczność
 - jeden crate może zawierać wiele modułów

Moduły w praktyce

```
mod math {  
    pub fn is_prime(n: u32) {  
        // implementation  
    }  
}  
  
fn main() {  
    math::is_prime(7);  
}
```

- `mod` tworzy moduł
- `pub` udostępnia elementy z modułu na zewnątrz
- ścieżka do symbolu w module to `math::is_prime`

Deklaracja `use`

- Deklaracja `use` skraca pełne ścieżki
 - `crate ::` odnosi się do bieżącego crate
 - `self ::` odnosi się do bieżącego modułu
 - `super ::` odnosi się do modułu nadrzędnego

Deklaracja `use`

```
mod primes {
    pub fn is_prime(n: u32) {
        // implementation
    }

    pub fn primes_in_range(range: std::ops::Range<u32>) → Vec<u32> {
        // implementation
    }

    pub mod generators {
        use super::is_prime;

        pub fn primes_sequence() → impl Iterator<Item = u32> {
            (2..).filter(|&n| is_prime(n))
        }
    }
}

fn main() {
    let is_seven_prime = primes::is_prime(7);
    let primes = primes::primes_in_range(2..20);
    let primes_seq: Vec<u32> = primes::generators::primes_sequence().take(10).collect();
}
```

Deklaracja `use`

```
mod primes {
    pub fn is_prime(n: u32) {
        // implementation
    }

    pub fn primes_in_range(range: std::ops::Range<u32>) → Vec<u32> {
        // implementation
    }

    pub mod generators {
        use super::is_prime;

        pub fn primes_sequence() → impl Iterator<Item = u32> {
            (2..).filter(|&n| is_prime(n))
        }
    }
}

fn main() {
    use crate::primes::{is_prime, primes_in_range};
    use crate::primes::generators::primes_sequence;
    let is_seven_prime = is_prime(7);
    let primes = primes_in_range(2..20);
    let primes_seq: Vec<u32> = primes_sequence().take(10).collect();
}
```

Pliki `main.rs` i `lib.rs`

- Pliki `main.rs` i `lib.rs` to tzw. *crate root* - pełnią specjalną rolę w definiowaniu binarnego i bibliotecznego crate'a.
 - `src/main.rs` dla binarnego crate
 - `src/lib.rs` dla bibliotecznego crate
- Rust zaczyna kompilację od tego pliku, traktując go jako moduł najwyższego poziomu (*crate-root*)
- Moduły w Rust są organizowane hierarchicznie, a plik `main.rs` lub `lib.rs` stanowi korzeń tej hierarchii.

Organizacja modułów w plikach

- Moduły można dzielić na pliki:
- Każdy moduł może być zdefiniowany:
 - w głównym pliku crate'a (np. `main.rs` lub `lib.rs`) deklaracją `mod nazwa_modułu;`
 - w osobnym pliku np. `primes.rs` - nazwa modułu to `primes`
 - lub w folderze z plikiem `mod.rs` - nazwa modułu to nazwa folderu
- Każdy moduł w osobnym pliku lub folderze musi być zadeklarowany w nadrzędnym module za pomocą `mod nazwa_modułu;`

Widoczność i organizacja

- Domyślnie symbole deklarowane w module są prywatne
- `pub(crate)` eksportuje wewnątrz crate'a
- `pub(super)` eksportuje do modułu nadrzędnego
- `pub` udostępnia globalnie z crate'a

Przykład modułów w plikach

src/lib.rs

```
mod primes; // load a module from primes.rs

pub use primes::{is_prime, primes_in_range}; // re-export functions from the primes module to the crate root
```

src/primes.rs

```
pub fn is_prime(n: u32) {
    // implementation
}

pub fn primes_in_range(range: std::ops::Range<u32>) → Vec<u32> {
    // implementation
}
```

- moduł `config` ładuje plik `config.rs`
- `pub use config::Settings;` ponownie eksportuje typ `Settings` na poziomie głównego modułu biblioteki.

Przykład modułu w folderze

src/fibonacci/mod.rs

```
pub mod numbers; // loads the module from src/fibonacci/numbers.rs
pub mod generators; // loads the module from src/fibonacci/generators.rs
```

src/fibonacci/numbers.rs

```
pub fn fibonacci(n: u32) → u32 {
    match n {
        0 ⇒ 0,
        1 ⇒ 1,
        _ ⇒ fibonacci(n - 1) + fibonacci(n - 2),
    }
}
```

src/fibonacci/generators.rs

```
pub fn fibonacci_sequence() → impl Iterator<Item = u32> {
    // implementation
}
```

Dzielenie dużego crate'a

- Rust umożliwia organizowanie kodu w moduły, co ułatwia zarządzanie dużymi projektami.
- Zagnieżdżone moduły pozwalają na lepszą organizację kodu i ukrywanie szczegółów implementacji.
- Zalecenia:
 - dziel duży crate na podmoduły i podfoldery
 - każdy folder reprezentuje przestrzeń nazw modułu
 - użyj `pub mod` , aby udostępnić zagnieżdżone moduły z korzenia
 - zachowaj szczegóły implementacji w prywatnych modułach, gdy to możliwe

Struktura folderów dla dużego crate'a

```
src/  
├── lib.rs      ← crate root  
├── primes.rs   ← moduł zadeklarowany w lib.rs  
└── fibonacci/  
    ├── mod.rs ← moduł zadeklarowany w lib.rs  
    ├── numbers.rs  
    └── generators.rs
```

src/lib.rs

```
pub mod primes;  
pub mod fibonacci;
```

To tworzy drzewo modułów:

```
crate  
├── primes  
└── fibonacci  
    ├── numbers  
    └── generators
```

Prelude w modułach

- Prelude w module to podmoduł (zazwyczaj nazwany `prelude`), którego jedynym zadaniem jest re-eksportowanie najczęściej używanych elementów danego modułu lub crate'a
- Jest to przydatne w dużych projektach, gdzie wiele modułów korzysta z tych samych elementów.
- Upraszcza to import często używanych elementów w wielu modułach.
- Biblioteka standardowa Rust ma swój własny prelude, który jest automatycznie dostępny w każdym module.

Prelude w modułach - przykład

- Definicja modułu `prelude` w `lib.rs` re-eksportuje najczęściej używane elementy z innych modułów.

`src/lib.rs` - crate-root biblioteki `numbers`

```
pub mod primes;
pub mod fibonacci;

pub mod prelude {
    pub use crate::primes::is_prime;
    pub use crate::primes::generators::primes_sequence;
    pub use crate::fibonacci::generators::fibonacci_sequence;
}
```

Import modułu prelude

- Importowanie prelude w innym module pozwala na korzystanie z najczęściej używanych elementów bez konieczności ich indywidualnego importowania.

```
use numbers::prelude::*;

fn example() {
    let primes = primes_sequence(10);
    let fib = fibonacci_sequence(10);
    println!("Primes: {:?}", primes);
    println!("Fibonacci: {:?}", fib);
}
```

Praktyczny układ aplikacji

```
my-app/
├── Cargo.toml
├── src/
│   ├── main.rs      # uruchomienie aplikacji
│   ├── lib.rs        # logika możliwa do testowania
│   ├── config.rs
│   └── storage/
│       ├── mod.rs
│       └── file.rs
└── tests/
    └── application.rs # testy integracyjne
```

- `main.rs` powinien być możliwie mały: odczytuje argumenty i uruchamia logikę
- kod domenowy umieszczaj w `lib.rs` i jego modułach
- testy integracyjne w katalogu `tests/` używają publicznego API biblioteki

Binarka korzysta z własnej biblioteki

src/lib.rs

```
pub mod config;
pub mod storage;

pub fn run() → Result<(), Box<dyn std::error::Error>> {
    let settings = config::load()?;
    storage::save(&settings.path, "ready"?);
    Ok(())
}
```

src/main.rs

```
fn main() → Result<(), Box<dyn std::error::Error>> {
    my_app::run()
}
```

- nazwa `my_app` pochodzi od pola `name` w `Cargo.toml` (myślniki zmieniają się na `_`)
- taki podział umożliwia testowanie `my_app::run()` bez uruchamiania procesu binarnego

Test integracyjny w katalogu tests/

tests/application.rs

```
use my_app::run;

#[test]
fn application_runs_successfully() {
    assert!(run().is_ok());
}
```

- każdy plik w tests/ jest kompilowany jako osobny crate testowy
- test importuje tylko publiczne elementy biblioteki, tak jak jej użytkownik
- cargo test uruchamia testy modułowe i integracyjne

Moduł jako plik i katalog

Struktura plików:

```
src/  
├─ lib.rs  
├─ network.rs  
└─ network/  
    ├─ http.rs  
    └─ tcp.rs
```

- `lib.rs` zawiera `pub mod network;`
- `network.rs` jest korzeniem modułu `network`
 - pliki w `network/` tworzą jego podmoduły
 - ten układ jest dziś często wybierany zamiast `network/mod.rs`

`src/network.rs`

```
pub mod http;  
pub mod tcp;  
  
pub fn init() {  
    println!("network initialized");  
}
```

Projekt z wieloma binarkami

```
src/  
├─ lib.rs  
├─ main.rs      # domyślnie: cargo run  
└─ bin/  
    ├─ import.rs  # cargo run --bin import  
    └─ report.rs  # cargo run --bin report
```

src/bin/report.rs

```
fn main() → Result<(), Box<dyn std::error::Error>> {  
    my_app::run()?;  
    println!("Report generated");  
    Ok(())  
}
```

- wspólną logikę zostaw w bibliotece, nie kopiuj jej między binarkami
- osobne binarki sprawdzają się dla narzędzi: importu, migracji i raportowania

Cargo.toml

Cargo.toml

- Plik konfiguracyjny `Cargo.toml` zawiera sekcje:
 - `[package]` zawiera metadane crate'a
 - `[dependencies]` definiuje zależności crate'a
 - `[dev-dependencies]` definiuje zależności używane tylko w testach i przykładach crate'a
 - `[build-dependencies]` definiuje zależności używane tylko w skryptach budowania
 - `[workspace]` definiuje wspólne ustawienia dla wielu crate'ów w jednym repozytorium
 - `[features]` definiuje opcjonalne funkcjonalności crate'a
 - `[lib]` definiuje ustawienia dla biblioteki crate'a, np. `name`, `path` i `crate-type`
 - `[[bin]]` definiuje ustawienia dla binarki crate'a, np. `name`, `path` i `crate-type`

[package] – metadane crate'a

- Najważniejsza sekcja, zawsze obecna.
- Zawiera:
 - nazwę crate'a
 - wersję
 - autora
 - licencję
 - opis
 - punkt wejścia (bin/lib)
 - edycję języka

```
[package]
name = "mycrate"
version = "0.1.0"
edition = "2021"
description = "Przykładowy crate"
license = "MIT"
```

[dependencies] – zależności zwykłe

- Zależności używane w kodzie produkcyjnym:
 - `features` włącza opcjonalną funkcjonalność crate'a, tutaj makro `derive`
 - Cargo dobiera kompatybilne wersje, pobiera i buduje zależności
- Domyślnie zależności są pobierane z crates.io
- Lokalne crate'y można dodawać za pomocą ścieżki, np. `path = "../config"`

■ `Cargo.toml`

```
[dependencies]
serde = "1.0"
tokio = { version = "1", features = ["full"] }
anyhow = "1"
config = { path = "../config" } # lokalny crate
```

■ `src/main.rs`

```
use serde::{Deserialize, Serialize};

#[derive(Debug, Deserialize, Serialize)]
struct User {
    name: String,
}
```

[dev-dependencies] – zależności testowe

- Zależności używane tylko w testach i przykładach:
 - nie są kompilowane do kodu produkcyjnego
 - można je dodawać za pomocą `cargo add --dev`

```
[dev-dependencies]
rand = "0.8"
tempfile = "3"
```

[features] – opcjonalne funkcje kompilacyjne

- Sekcja `[features]` pozwala definiować zestawy funkcji, które mogą być włączane lub wyłączane podczas kompilacji.
- Cargo.toml

```
[features]
default = ["logging"]
logging = []
experimental = ["serde"]
```

- Włączenie funkcji w kodzie aplikacji:

```
#[cfg(feature = "logging")]
mod logging;
```

```
#[cfg(feature = "logging")]
pub fn log(msg: &str) {
    println!("[LOG] {msg}");
}

pub fn run() {
    #[cfg(feature = "logging")]
    println!("Startuję...");

    // reszta kodu
}
```

[lib] – konfiguracja crate'a bibliotecznego

- Sekcja `[lib]` pozwala określić ustawienia dla crate'a bibliotecznego, takie jak nazwa, ścieżka do pliku źródłowego i typ crate'a.

```
[lib]
name = "mycrate"
path = "src/lib.rs"
crate-type = ["rlib", "cdylib"]
```

[bin] – konfiguracja crate'a binarnego

- Sekcja `[bin]` pozwala określić ustawienia dla crate'a binarnego, takie jak nazwa, ścieżka do pliku źródłowego i typ crate'a.
- Można mieć wiele binarek w jednym crate'cie:

```
[[bin]]  
name = "server"  
path = "src/bin/server.rs"
```

```
[[bin]]  
name = "client"  
path = "src/bin/client.rs"
```

- Uruchamianie binarek:

- `cargo run --bin server`
- `cargo run --bin client`

[workspace] – konfiguracja workspace

- Gdy projekt składa się z wielu crate'ów, można je zorganizować w workspace
- Przykładowa struktura workspace:

```
[workspace]
members = ["apps/cli", "crates/core", "crates/config"]
resolver = "2"
```

- Workspace współdzieli `Cargo.lock` i katalog `target/`
- Pozwala na centralne zarządzanie zależnościami i wersjami crate'ów
- `resolver` określa strategię rozwiązywania zależności w workspace (np. `"2"` dla nowego algorytmu).

Workspace: wiele crate'ów, jedno repozytorium

Główny `Cargo.toml`

```
[workspace]
members = ["apps/cli", "crates/core", "crates/config"]
resolver = "2"
```

```
[workspace.dependencies]
serde = { version = "1", features = ["derive"] }
```

`apps/cli/Cargo.toml`

```
[dependencies]
core = { path = "../..crates/core" }
serde = { workspace = true }
```

- workspace współdzieli `Cargo.lock`, katalog `target/` i definicje zależności
- wspólne wersje w `[workspace.dependencies]` zapobiegają przypadkowym rozjazdom

Codzienna praca z Cargo

```
cargo new my-app          # binarny crate
cargo build                # kompilacja crate'a
cargo run                  # uruchomienie crate'a
cargo new --lib my-lib     # biblioteczny crate
cargo add serde --features derive # dodanie zależności z funkcją derive
cargo add --dev tempfile  # dodanie zależności tylko do testów i przykładów
cargo check                # szybka kontrola typów
cargo test --workspace    # uruchomienie testów we wszystkich crate'ach w workspace
cargo update -p serde     # aktualizacja wybranego crate'a
cargo tree                 # drzewo zależności
```

- `cargo add` zapisuje zależność w poprawnej sekcji `Cargo.toml`
- commituj `Cargo.lock` w aplikacjach i workspace'ach, aby zachować powtarzalne buildy
- biblioteki zwykle nie publikują `Cargo.lock`

Podsumowanie

- moduły organizują kod wewnątrz crate
- crate to jednostka kompilacyjna i dystrybucyjna
- `pub` , `mod` , `use` i ścieżki definiują widoczność
- `Cargo.toml` zarządza zależnościami, funkcjami i targetami
- workspace porządkuje większy projekt złożony z wielu crate'ów

Dziękuję za uwagę

krystian.piekos@infotraining.pl